

Cooperative parallel grouping genetic algorithm for the one-dimensional bin packing problem

Tayfun Kucukyilmaz^{a,*}, Hakan Ezgi Kiziloz^b

^a TED University, Ankara, Turkey

^b University of Turkish Aeronautical Association, Ankara, Turkey

ARTICLE INFO

Keywords:

1D bin-packing
Grouping genetic algorithm
Parallel evolutionary algorithms
Search space diversity
Migration topology
Assimilation strategy

ABSTRACT

Evolutionary algorithms have been reported to be efficient metaheuristics for the optimization of several NP-Hard combinatorial optimization problems. In addition to their ability to solve difficult and complex problems in reasonable execution times, parallelized versions of evolutionary algorithms are reported to explore and exploit the problem search space more effectively than their sequential counterparts. The *Island Model*, where the population of a given run is divided into semi isolated subpopulations, is a popular parallelization approach for evolutionary algorithms such as Grouping Genetic Algorithms (GGA). Although the nature of GGAs is very suitable for coarse-grained parallel processing, designing an Island-parallel model for them is not a straightforward task. Selecting the communication topology, deciding migration and assimilation strategies, adjusting the migration rate and frequency, and using efficient diversification techniques are some of the important issues that needs to be covered in a successful Island-parallel Model. In this study, we propose a novel, scalable Island-parallel GGA (IPGGA) for the well-known combinatorial optimization Problem 1D Bin-Packing (1DBPP). We provide a thorough experimental evaluation of the parallel model and report significant improvements on the Hard28 problem instances by outperforming the state-of-the-art genetic algorithms. Additionally, we analyze and evaluate the parallelization parameters of IPGGA with an emphasis on problem search-space diversity and report several interesting results.

1. Introduction

One-dimensional Bin-Packing Problem (1DBPP) is a challenging NP-Hard combinatorial optimization problem. In 1DBPP, each item is associated with a single cost and the objective is to pack finite number of items into the minimum number of bins (Alvim, Ribeiro, Glover, & Aloise, 2004; Gary et al., 1979; Johnson, Demers, Ullman, Garey, & Graham, 1974). There are numerous real life applications and adaptations of 1DBPP, where the items of interest are subject to various constraints (Coffman, Garey, & Johnson, 1996; Fleszar & Charalambous, 2011; Fleszar & Hindi, 2002; López-Camacho, Terashima-Marín, Ochoa, & Conant-Pablos, 2013). Formally (Martello & Toth, 1990), the binary integer programming model for 1DBPP is defined as:

$$\begin{aligned} & \text{minimize} && \sum_{k=1}^K y_k \\ & \text{subject to} && \sum_{k=1}^K x_{ik} = 1, \quad \forall i \in \mathcal{I} \\ & && \sum_{i=1}^n w_i x_{ik} \leq W y_k, \quad \forall k \in \mathcal{K} \\ & && y_k \in \{0, 1\}, \quad \forall k \in \mathcal{K} \\ & && x_{ik} \in \{0, 1\} \quad \forall i \in \mathcal{I}, \quad \forall k \in \mathcal{K} \end{aligned}$$

where n is the number of items, w_i is the weight of item i , W is the bin capacity, $\mathcal{I} = \{1, \dots, n\}$ is the set of items, $\mathcal{K} = \{1, \dots, K\}$ is the set of bins, and the variables x_{ik} and y_{ik} are defined as:

$$x_{ik} = \begin{cases} 1 & \text{if item } i \text{ is assigned to bin } k, \\ 0 & \text{otherwise} \end{cases}, \quad y_k = \begin{cases} 1 & \text{if bin } k \text{ is used,} \\ 0 & \text{otherwise} \end{cases}$$

Although 1DBPP with small number of items (up to 50) can be evaluated exactly with brute-force algorithms, large problem instances of the 1DBPP cannot be solved trivially due to the problem's intractable

* Corresponding author.

E-mail addresses: tayfun.kucukyilmaz@tedu.edu.tr (T. Kucukyilmaz), hekiziloz@thk.edu.tr (H.E. Kiziloz).

nature. Therefore, metaheuristic approaches such as genetic algorithms (GA), particle swarm optimization, tabu search, and minimum bin slack have been widely used to approximate the result of this important problem almost-optimally (Buljubai & Vasquez, 2016; Cantu-Paz, 2000; Dokeroglu & Cosar, 2014, 2016; Fernández, Gil, Baños, & Montoya, 2013; Holland, 1992; Luque & Alba, 2011). In this work, we concentrate our efforts on a specific subset of GA known as grouping genetic algorithms (GGA) for solving the 1DBPP.

The GGA is a heavily modified version of GA, specifically designed to solve complex grouping problems. In the original article (Falkenauer, 1996a), the authors specifically tailored GGA for solving the Bin Packing Problem (BPP). The fundamental difference of GGA from GA is that, each candidate solution is divided into two parts: an *element* section, which denotes the items of interest and a *grouping* section, which denotes the clusters that the items are intended to be partitioned into. Formally, let n items be divided into k clusters, where x_i is the assignment variable of item i , and $\mathcal{N}_k$ denotes the set of items in the k^{th} bin; a candidate solution will be defined as:

$$S = \{I_1, I_2, I_3, \dots, I_n | G_1, G_2, \dots, G_k\}$$

where, $I_i = G_k \Leftrightarrow x_i \in \mathcal{N}_k$

Also note that in the GGA model, although the length of the element section is fixed for a candidate solution, group section's length is not fixed. This allows GGA be a suitable algorithm for 1DBPP, where the number of target bins is not known apriori.

Although most of the state-of-the-art metaheuristic algorithms that have been proposed to solve 1DBPP in the literature are designed to run on a single processor, there is an undeniable interest for using high performance computing opportunities that are offered by the recent parallel computation technologies (Andre & Koza, 1996; Tomassini, 1999; Tufts, 1995) due to growing problem sizes. One of the popular approaches for parallelizing such evolutionary algorithms is the *Island Model* (Ruciński, Izzo, & Biscani, 2010). The Island Model can be considered as a coarse-grained parallelization of a problem, where the global population is divided into semi-isolated subpopulations called *demes*. Each deme in this approach is usually assigned to one of the computational units in the computing infrastructure. Although each computational unit is only responsible on its assigned population, emigration among these subpopulations is usually considered as an operation with a positive impact, allowing a better diversified population and better exploitation of the problem-space.

In this study, we intend to improve the performance of the state-of-the-art GGA by applying scalable high performance parallel algorithms (Falkenauer, 1994, 1996b; Mitchell, 1998) using an Island parallelization model (Ruciński et al., 2010). To this end, we analyze the effects of both GGA parameters such as population size and number of generations, and parallelization-specific parameters such as communication topology, migration rate and frequency, and possible emigration and assimilation strategies with respect to their effects on result performance and scalability. We evaluate communication topologies in further detail, examining both static and dynamic communication topologies, showing that the proposed parallelization is in fact scalable. We also aim to provide a thorough discussion on the parallelization parameters in respect to their effects on diversification in the problem-space.

The organization of the paper is as follows: in Section 2, related studies for the state-of-the-art GGA and parallel adaptations are summarized. In Sections 3 and 4, components of the adopted sequential GGA and the proposed Island-Parallel GGA (IPGGA) are introduced and potential communication topologies that can be used in the parallel model are described. A discussion on the experimental results are presented in Section 5. Finally, concluding remarks are provided in Section 6.

2. Related work

The literature reveals that Bin Packing is a very well-studied

combinatorial optimization problem. The classic BPP, specifically One-Dimensional BPP, is defined as a special case of the well-known Cutting Stock Problem and first studied by Ullman (1971). To date, many variations and modifications of the classic Bin Packing Problem found practical uses in real life and examined by a vast literature. Multi-dimensional Bin Packing (Beyaz, Dokeroglu, & Cosar, 2015; Christensen, Khan, Pokutta, & Tetali, 2017), linear packing (Berghammer & Reuter, 2003; Błażewicz & Ecker, 1983), packing by weight and cost (Epstein & Levin, 2012; Leinberger, Karypis, & Kumar, 1999), bin packing with rejection (Bein, Correa, & Han, 2008; Epstein, 1999), bin packing with dynamic item sizes (Correa & Epstein, 2008; Stolyar & Zhong, 2016), dynamic bin constraints (Maiza, Radjef, & Sais, 2016), and online packing (Boyar, Kamali, Larsen, & López-Ortiz, 2016) are some of the popular and well-known variations of the classic BPP. There are also several excellent surveys (Coffman, Csirik, Galambos, Martello, & Vigo, 2013; Coffman, Galambos, Martello, & Vigo, 1999; Martello & Toth, 1990) in the literature that summarize many of the standing work.

Apart from brute-force approaches, several works evaluate exact or high-quality solutions for the BPP such as Tabu Search (Alvim et al., 2004), Branch-and-Bound algorithm (Scholl, Klein, & Jürgens, 1997), mixed integer programming models (Arbib & Marinelli, 2017), Neural Networks (de Almeida & Steiner, 2015), and Cuckoo Search Algorithm (Zendaoui & Layeb, 2016). Two recent articles present the state-of-the-art techniques on the subject, presenting significant improvements for the BPP. In Buljubai and Vasquez (2016), the authors propose a multi-step Tabu search approach for finding a near-optimal solution for the BPP. In their work, first a partial solution is found using dynamic programming and enumeration, which induces a partitioning over the item set: assigned items and non-assigned items. The proposed algorithm then attempts to pack the non-assigned items into the remaining bins. Results show significant improvements on the well-known Hard28¹ dataset, finding 27/28 optimal solutions in an acceptable amount of time. Brandão and Pedroso (2016) attempts to find an exact solution to the BPP using an arc-flow formulation. The work mainly focuses on pruning the resulting arc-flow graph in order to reduce the execution time without losing quality. In both works however, the authors also acknowledged the fact that the proposed methods may not scale well with larger datasets or work well on datasets with specific characteristics. Additionally, it is worth pointing out the fact that since both methods use data-wide global information in their optimization, using parallelization in order to scale the proposed methods for larger datasets would not be a viable option. For larger problem sizes, algorithms that are inherently more suitable to parallelism also have merits beyond their result quality.

Genetic Algorithms (GA) are one such class of metaheuristic approaches that has been extensively evaluated for the BPP (Stawowy, 2008; Thomas & Chaudhari, 2014). In fact, possible parallel adaptations of GAs have been investigated in the literature for the last several decades (Grefenstette, Gopal, Rosmaita, & Gucht, 1985). To date, numerous studies have been conducted, several excellent books (Cantu-Paz, 2000; Luque & Alba, 2011; Ribeiro & Hansen, 2002) and reviews (Alba & Troya, 1999; Konfrst, 2004; Tomassini, 1999) have been published on parallel GAs. We should note here that the recent attraction GAs get is not due to their superior result quality or running time. The literature shows that there are already exact and heuristic methods that perform superior to GAs in both aspects, although most such algorithms tend to use a global-view of the problem space and may not be very suitable for large scale parallelization. GAs on the other hand are very suitable for parallelization and may perform reasonably well with very large problem sizes where other methods cannot. In this work, our aim

¹ Each of the benchmark instances in the dataset also contain the optimal number of bins, which is established by L2 lower bound presented in Martello and Toth (1990) using branch-and-bound estimations.

is to provide a scalable framework suitable for any GA model and analyze the parallelization parameters along with GA parameters.

Due to their ease of management, result quality (Cantu-Paz, 2000), and ability to alleviate the bloat phenomenon (Ruciński et al., 2010), island-parallel approaches attracted a lot of attention as a parallelization strategy of GAs in the literature. Nalepa and Blocho (2015) investigate the impact of selecting a co-operation scheme for the parallel memetic algorithm to solve the NP-hard vehicle routing problem with island-parallel model. In order to solve the well-known quadratic assignment problem, Tosun, Dokeroglu, and Cosar (2013) proposed an island-parallel GA approach using a master-slave communication topology. A recent study (Liu & Wang, 2015) described a scalable coarse-grained parallel algorithm for Generalized Assignment Problem (GAP). Dokeroglu and Cosar (2014) presented a set of robust and scalable hybrid parallel algorithms that take advantage of parallel computation techniques such as evolutionary grouping genetic metaheuristics, and bin-oriented heuristics to obtain solutions for large scale one-dimensional BPP instances. Parallel models have also been applied in several studies to adaptations of BPP, such as multi-dimensional BPP (Bożejko, Uchroński, & Wodecki, 2010; Dokeroglu & Cosar, 2014; Fernández et al., 2013; Kröger, Schwenderling, & Vornberger, 1990; Pargas & Jain, 1993; Segura, Segredo, & León, 2011). Another recent study by Ozcan, Dokeroglu, Cosar, and Yazici (2016) investigates the performance of GGA over GPU and concluded that, using GPU for complex tasks such as crossover and mutation improves execution time of the algorithm significantly.

With the introduction of parallel models, there has also been a large interest shift in the research on evolutionary algorithms. Due to their indisputable importance on execution time and result quality, numerous works have investigated the impacts of parallelization parameters such as population size (Fernández, Tomassini, Punch, & Sánchez, 2000), communication topologies (Andre & Koza, 1996; Fernández et al., 2000; Ruciński et al., 2010; Schwehm, 1996), different communication strategies between processes (usually referred to as *migration*), migration rate (Andre & Koza, 1996; Fernández et al., 2013; Juille & Pollack, 1995; Schwehm, 1996), migration frequency (Andre & Koza, 1996; Fernández et al., 2013; Juille & Pollack, 1995; Schwehm, 1996), and synchronization in communication (Fernández, Galeano, & Gómez, 2002).

In this work, our aim is threefold: first, we aim to present a scalable island-parallel model for the state-of-the-art GGA. To this end, we will present a parallel adaptation of the state-of-the-art GGA models. Second, in the island-parallel model, we aim to investigate the communication topologies and strategies more thoroughly, which we feel still has uncovered potential. Also, since many of the most recent studies concentrate on the importance of problem-space diversity in terms of result quality, we want to explore the effects of parallelization on diversification. Third, we aim to provide more insight on different parallelization parameters on an island-model GGA and their effects on result quality.

3. The sequential grouping genetic algorithm framework

In general, a GA is a metaheuristic approach that aims to find solutions to complex problems in an iterative fashion. Each iteration in a GA is also referred to as a *generation* (Steps 2 and 3 below). A run of a GA involves the following steps:

1. **Initial Population Creation:** Each run of a genetic algorithm starts with creation of an initial population, where each population instance represents a candidate solution represented as a set of features named as chromosomes.
2. **Fitness Calculation:** At each iteration, for each instance in the population a fitness value is calculated. In principle, the calculated fitness values represent how close the candidate solutions are to the objective.

3. **Darwinian Selection:** At each iteration, some candidate solutions are selected probabilistically (preferably based on the calculated fitness values) to participate in some predefined genetic operations.
4. **Iteration:** The second and third steps are iterated until a candidate solution converges to the objective or a predefined number of iterations, also referred to as generations, have been completed.

In this approach, the representation of a candidate solution and the definition of genetic operations that will be performed on candidate solutions form the basis of the applied algorithm. Here, we will explain these briefly.

3.1. Chromosome representation

The chromosome representation, also referred to as the genetic encoding, of a GA determines how one candidate solution is represented. In our work, a group-based chromosome representation, also referred to as Falkenauer-style chromosome, is adopted since it is proven to be highly superior to other representations in terms of computational and result performance for the BPP. The reason for adopting this representation is twofold: First, group-based chromosomes allow different candidate solutions to have different chromosome sizes, which is more suitable for cases where the target number of bins is unknown. Second, this representation allows an inherent solution to an important performance bottleneck in GAs called *degeneration*, which leads to inefficient coverage of the search space where the same configuration of groups is explored repeatedly (Radcliffe, 1991).

3.2. Initial population generation

The first operation in the run of a GA is the generation of a set of candidate solutions. It is known that the quality of the initial population greatly effects both execution time and result quality of a GA run. In Quiroz-Castellanos et al. (2015), the authors argue that an intelligent selection of initial population would reduce the number of iterations considerably and propose a $FF_{\tilde{n}}$ heuristic for BPP. In this two-step heuristic, first largest $\tilde{n}$ items are placed into separate bins; second, the remaining items are placed into remaining spaces in the bins in a first-fit manner. In their work (Quiroz-Castellanos et al., 2015), the authors report considerable improvements on a variety of benchmark datasets. Due to its superior result quality in the literature, we also adopt the same strategy for creating the initial population and use the $FF_{\tilde{n}}$ heuristic in this study.

3.3. Fitness function

In this work, the fitness of a candidate solution is evaluated using a cost function that is proposed by Falkenauer (1994). Eq. (1) summarizes this cost function, where nb is the number of bins, F_i is the sum of weights of the elements packed into the bin $i \in 1..nb$, c is the bin capacity, and k is a heuristic exponential factor. The value k expresses a concentration and provides a favoritism for the almost-full bins in comparison to less filled ones. In Stawowy (2008), it is reported that $k = 4$ gives the best results. Thus, for the purpose of fitness calculations we have adopted the same formulation with proper adherence to the literature, using $k = 4$.

$$FF = \sum_{i=1}^{nb} \left(\frac{F_i}{c} \right)^k \quad (1)$$

3.4. The Darwinian selection

In this work, we have adopted three different selection operators and apply these operators to each generation of the presented GGA in the following order: First, a crossover operator is applied allowing the

candidate solutions to interact at the gene level to produce new offspring. Second, a mutation operator is applied for facilitating the trade-off between exploration and exploitation of the search space. Finally, a replacement heuristic involving both parent and children solutions is applied to determine the population for the next generation run. These steps are repeated in the given order at every generation of the GGA in this study. Next, we briefly describe these operators.

3.4.1. The crossover operation

The crossover operation constitutes to the reproductive process in a GA. The crossover operator takes two candidate solutions and recombines them in order to produce a child solution, where the child inherits the chromosomes (solution segments) from both parents.

For the purpose of GGA, where both items and bins are represented as chromosomes, a more suitable definition of a crossover operator is a *gene-level crossover*: A high quality solution in a GGA representation can be characterized as a solution containing many bins that are almost full. Informally, this implies that the bin-fullness is adopted as the representative for the solution quality. In gene-level crossover, the aim is to preserve, or give higher survivability to the bins that are fullest. In our study, we have adopted a gene-level crossover operator described in Quiroz-Castellanos et al. (2015). First, n random solutions are selected from the population's best $2n$ candidates with respect to their fitness values, and n more solutions are selected among all candidates in the population. These two groups are then used as parents in the crossover operation which involves several steps.

Initially, the genes of each selected solution are sorted in descending order and the selected solutions are paired in two's for the crossover operator. Starting from the fullest bin for each selected pair, the bin fullness is compared bin by bin, and the fullest bin is inherited by the child, followed by the bin in the other parent. After all bins are inherited by the child, the contents of each bin are compared and bins with duplicate items are removed from the child. At the end, if there are any items that are not placed in any of the remaining bins, FFD packing heuristic is used to redistribute them in the offspring solution.

3.4.2. The mutation operation

The mutation operation in a GA introduces random changes to the chromosomes of the population. In the GGA representation, the mutation operator removes some of the bins from the solution, re-inserting the free items to the already existing bins or generating new bins if need arises (Falkenauer & Delchambre, 1992). In this work, during the mutation operation, the emptiest bins are selected for elimination, removed from the gene pool, and the items are placed into the existing bins using a $FF_{\tilde{h}}$ heuristic. However, we have also adopted a second rearrangement heuristic described in Quiroz-Castellanos et al. (2015) during the re-insertion of the free items. In this approach, the free items are also compared with the already-binned items. If swapping these item pairs still produce a viable solution, i.e. solutions where bin capacities are not exceeded, the already-binned items and the free items are swapped. The swap operation is performed regardless whether there is fitness improvement or not. The rationale of using this rearrangement heuristic is that it promotes a much diverse solution population allowing exploration of different item permutations in the solution space.

3.4.3. The replacement heuristic

After the crossover and mutation operations, the population has changed and expanded with the introduction of new children (or

mutated parents). The replacement heuristic constitutes to the selection process of the population for the next generation of the GA. In this work, the child solutions of the crossover process are always inserted to the population using a similar method described in the crossover operation: Half of the child solutions are inserted, replacing random candidate solutions, and the remaining child solutions are inserted replacing either candidate solutions with similar fitness values or the worst solutions if no solutions with similar fitness values can be found.

4. Island Parallel GGA (IPGGA) framework

In this section, we provide a detailed summary on the main elements of the proposed Island Parallel Grouping Genetic Algorithm (IPGGA) for the 1DBPP. In the proposed IPGGA, the intra-processor communication is handled using Message Passing Interface (MPI) libraries in C++.

The execution process of an island-parallel GGA is quite similar to the execution process of a sequential GGA. Conceptually, IPGGA can be viewed as a set of sequential GGA tasks running in parallel: At all computational nodes, a set of candidate solutions are generated randomly and core genetic operations are applied to these populations in an iterative manner. The iterative task is repeated until the algorithm converges to an optima or until a fixed number of iterations (i.e. generations) has passed. In this view, each island can be conceptualized as an isolated GGA task.

Although it is possible to view island-parallel model as a set of sequential tasks, multiple parallel executions of the same GA allow an additional beneficial operation called *migration*, where separate executions communicate with each other, possibly sharing solutions in order to achieve faster/better results. It has been shown in literature that, giving intelligent migration decisions can lead to best results (Andre & Koza, 1996). In this study, we also evaluate several migration alternatives and their repercussions on result quality thoroughly.

Algorithm 1 summarizes the execution process of an IPGGA. Assuming there are n computational nodes, each of these nodes are perceived as an island. First, an initial population is created randomly and independently at each island in a parallel fashion. This operation is denoted in Algorithm 1 at lines 4–5. Lines 6–8 of Algorithm 1 constitute to a generation of the island-parallel GGA. In this step, each island performs local operations such as fitness calculation and Darwinian selection operators and compute their modified populations for the next generation run. The operations in this step and their inherent order are described in more detail in Section 3, Steps 2 and 3. Lines 9–30 constitute to the global communication operation in the island-parallel approach, where computational nodes communicate with each other to immigrate and emigrate prospective population instances via a migration operator. Please note here that, the communication operation is dependant on the adopted communication topology and handled separately for each of the different topologies. For example, lines 13–19 describes how communication is handled for a 2_Clique communication topology (Fig. 2a), lines 20–23 for a 4_Clique communication topology (Fig. 2b), and lines 24–27 for an OOPC communication topology (Fig. 2c). Finally, line 31 denotes the assimilation process of the communication step, where immigrant populations are merged with the population of the particular island. As a final note, we would also like to point out here that the computational bottleneck for GGA is the fitness calculation and communication usually only has a small contribution to the total execution time (Ruciński et al., 2010).

Algorithm 1. Island-Parallel Grouping Genetic Algorithm (IPGGA)

```

1  n: processor_count;
2  p: population_size;
3  t: communication topology;
4  for i:=1 to n do
5    Generate_Initial_Population(p);

6  for i:=1 to n do
7    while (i:=1 to max_number_of_generations) OR (solution_converges) do
8      Generation(p); // Section 3, steps 2 and 3
9      //Migration operator. Send(receive) candidates to(from) other processors
10     m: migration rate;
11     i=0: current communication round;
12     if (i % Migration_Frequency == 0) then
13       if (t == 2_CLIQUE_TOPOLOGY) then
14         if (processor_rank % 2 == 0) then
15           pm = Receive the best chromosome from (rank + 1);
16           Send the global best chromosome to (rank + 1);
17         else
18           Send the local best chromosome to (rank - 1);
19           pm = Receive global best chromosome (rank - 1);
20       if (t == 4_CLIQUE_TOPOLOGY) then
21         Send the global best chromosome to (rank + (i % 4)) % n;
22         pm = Receive the global best chromosome to (rank + (-i % 4)) % n;
23         i++;
24       if (t == OOPC_TOPOLOGY) then
25         Send the global best chromosome to (rank + i) % n;
26         pm = Receive the global best chromosome to (rank - i) % n;
27         i++;
28     .
29     .
30     . // Migration operator for other communication topologies
31     Assimilate(pm);

```

It is important to note here that Algorithm 1 is a very generic depiction of an IPGGA: Algorithm 1 does not identify several important aspects of the migration operation such as the ideal rate and frequency of the migration, the migration size, the impact of migration on the already existing population (i.e., what solutions should be sent to other processors/removed from the already existing population to accommodate the immigrant population?), the communication pattern between islands, and how to give a termination decision. In the rest of this section, we aim to address how communication could be performed between islands as a part of the design in detail, and discuss the other aspects of GGA in Section 5 along with accompanying experiments.

Among all aspects of an island parallel GGA, perhaps the most influential one is the adopted communication topology. Had there been no communication between islands, it is easy to see that a parallel realization of GGA would degrade to just a set of independent sequential tasks. The communication allows prospective candidate solutions to be promoted, and the communication topology defines how computational nodes share prospective candidate solutions between local island populations via the migration operation. Due to this fact, the diversification of solution space on localized subpopulations depends largely on the selected communication topology.

In this work, we have divided and examined the possible

communication topologies in two groups: static and dynamic communication topologies. In static communication topologies, the edges between vertices always incur a migration operation. Moreover, these predefined communication channels cannot change until a GGA run finalizes. For this study, we have adopted four different static communication topologies with different characteristics that are widely used in parallel and distributed computing. These topologies are: Master-slave (Star), ring, 2D toroidal grid, and hypercube communication topologies. Fig. 1 presents the graphical depictions of these topologies. Note here that, in terms of graph analysis, the proposed topologies have very different realizations in terms of diameter, edge-to-vertex ratio, and communication balance.

Fig. 1a represents the graphical depiction of master-slave, also referred to as star, communication topology. Among all examined topologies, master-slave topology has the lowest diameter and has a constant edge-to-vertex ratio which makes it a practical choice for small-sized, fine-grained parallel solutions. Also note here that, during the migration operation all best candidates are accumulated by the center node and then distributed to other nodes, possibly effecting the diversification of global population adversely.

Fig. 1b represents the graphical depiction of the ring communication topology. In this topology, during the communication phase (Lines

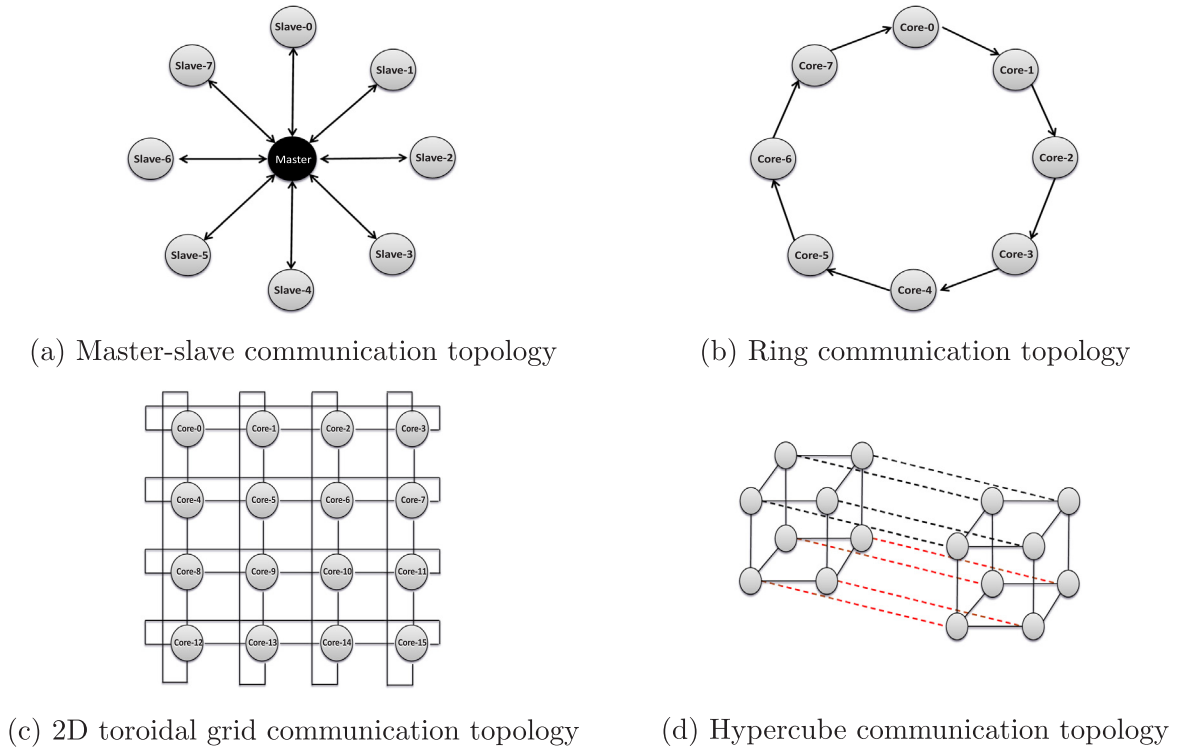


Fig. 1. Static communication topologies for island parallel GGA.

10–31 in Algorithm 1) each island can only communicate with its leftmost or rightmost neighbor. Hence, among all topologies, the ring topology has the highest diameter and the communication load of all processors is almost evenly distributed. In terms of island parallel GGA model, the high diameter of the ring topology indicates that the propagation of the best candidates would be slow, but the population diversity would be preserved better.

Both 2D toroidal grid and hypercube communication topologies are good compromises between the graph diameter and edge-to-vertex ratio: having the diameter size $\sqrt{n}$ and $\log(n)$ for a graph with n vertices respectively. For an island parallel GGA model, both topologies would allow a fast propagation of better candidate solutions while the communication patterns between computational nodes are distributed in a roughly balanced fashion. Consequently, one can argue that 2D toroidal

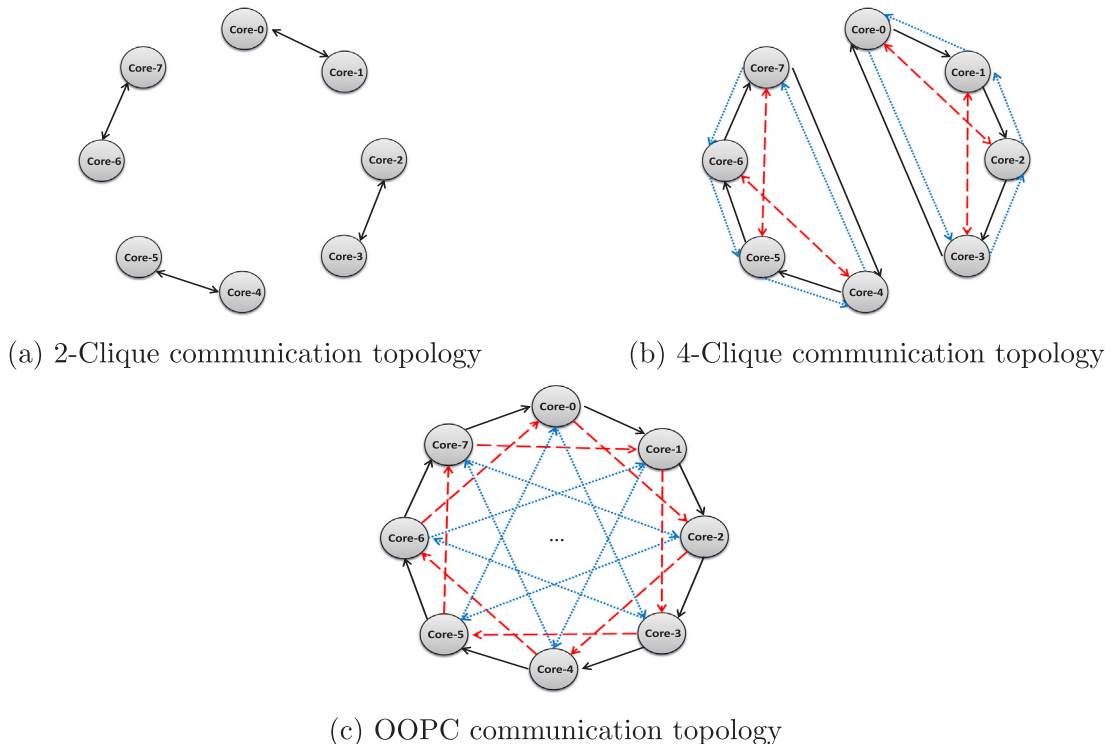


Fig. 2. Dynamic communication topologies for island parallel GGA.

Table 1
Information about the 1DBPP instances.

Problem instance	# instances	Item weights	Bin capacity (c)
u_120	20	[20,100]	150
u_250	20	[20,100]	150
u_500	20	[20,100]	150
u_1000	20	[20,100]	150
Data Set 1	720	[1,100]	{100,120,150}
Data Set 2	480	[3,9] items at each bin	1000
Data Set 3	10	[20,000,35,000]	100,000
Hard28	28	[1,800]	1000

Table 2
Default parameters for IPGGA.

Parameter	Value
Ratio of individuals to combine	0.2
Ratio of individuals to mutate	0.83
The ratio of individuals in the elite set	0.1
The ratio of bins to eliminate in the mutation operator (for non-cloned)	k = 1.3
The ratio of bins to eliminate in the mutation operator (for cloned)	k = 4
Seed number	The processor rank in MPI environment

grid and hypercube communication topologies would provide a better balance between exploration and exploitation of problem search space for a parallel GGA. It is also known in the literature that these two topologies are more scalable than ring and master-slave communication topologies (Ruciński et al., 2010).

Insight of the above discussions is that; the diameter of the communication topology has both pros and cons for an island parallel GGA approach. A high-diameter topology, by propagating the best candidate solutions slower, has the advantage of having islands more isolated, effectively providing a natural solution to the bloat phenomenon (Ruciński et al., 2010). On the other hand, a low-diameter topology propagates the best candidate solutions faster, allowing all computational nodes work on elite subsets of the solution space. One natural question that would immediately arise is “is it possible to use a high diameter model that changes the communication patterns in order to facilitate both advantages?”.

As discussed previously in Section 2, communication topologies that change the communication patterns are almost completely neglected in the literature. As an attempt to examine such networks, we evaluate a second set of topologies which we call *dynamic* communication topologies. The term dynamic here refers to the fact that there is no regular communication pattern among the processors, and the communication channels between computational nodes may change during a GGA run. To this end, we propose three dynamic communication topologies. These topologies are: 2-Clique, 4-Clique, and One-to-one personalized communication (OOPC) topologies. Fig. 2 depicts the graphic illustrations of these topologies for 8 computational nodes.

Figs. 2a and b represents two two-level hierarchical dynamic communication topologies. The 2-Clique and 4-Clique topologies severely

Table 4
The effects of population size per island on result quality.

Population size per island	u_120	u_250	u_500	u_1000	Data Set 1	Data Set 2	Data Set 3	Hard28
200	20	20	20	20	720	480	10	15
300	20	20	20	20	720	480	10	8
400	20	20	20	20	720	480	10	9
500	20	20	20	20	720	480	10	16
1000	20	20	20	20	720	480	10	19
2000	20	20	20	20	720	480	10	17
4000	20	20	20	20	720	480	10	19
6000	20	20	20	20	720	480	10	20

restrict the communication among islands: in the 2-Clique communication topology only island pairs and in the 4-Clique communication topology only island quartets are allowed to communicate with each other, while any other communication is completely disallowed. Informally, this communication model generates isolated archipelagos; sets of isolated islands. We believe that the comparison of the results of static and dynamic communication topologies may provide valuable insight on the facets of the migration operator.

The 2-Clique topology depicted in Fig. 2a is an extreme case of archipelago model; although being a dynamic approach, the communication pattern never changes. The other extreme, as depicted in Fig. 2c is the one-to-one personalized communication topology. Here, the hierarchical archipelago consists of every island in the topology. OOPC can also be considered as having a ring topology, with constantly shifting communication assignments. Two main benefits of this network are its diameter and message propagation rate: the diameter of the graph is n for n vertices, while the propagation rate of a message is $\log(n)$. That is, the total incurred communication in this topology is as low as a static ring topology while propagating a candidate solution from one computational node to another only takes $\log(n)$ hops, just like hypercube topology.

5. Analysis and experimental results

We ran our tests on a high performance computer cluster consisting of 4 AMD Opteron 6386SE processors, each having 16 cores, containing a total of 64 CPUs. The experimental framework has 264 GB of RAM installed, which corresponds to a large enough memory space to run our tests.

During our experiments, we have evaluated our models using three sets of problem instances: Falkenauer U (u_120, u_250, u_500 and u_1000) (Falkenauer, 1994, 1996b), Scholl (Data set 1, Data set 2 and Data set 3) (Scholl et al., 1997), and Hard28 (Belov, Scheithauer, & Mukhacheva, 2008). Details of these datasets are given in Table 1. Falkenauer U and Scholl datasets are considered relatively less challenging and are solved perfectly using various methods in the literature (Quiroz-Castellanos et al., 2015). Hard28 dataset is generally considered the most challenging bin packing benchmark. During our evaluations, experiments on Falkenauer U and Scholl datasets reveal the fact that many of the problem instances on these datasets require very small number of generations for finding an optimal solution and hence

Table 3
The effects of population size on result quality in a sequential run.

Population size	u_120	u_250	u_500	u_1000	Data Set 1	Data Set 2	Data Set 3	Hard28
200	20	20	20	20	720	480	10	12
500	20	20	20	20	720	480	10	14
1000	20	20	20	20	720	480	10	14
2000	20	20	20	20	720	480	10	14
4000	20	20	20	20	720	480	10	14
6000	20	20	20	20	720	480	10	14

Table 5

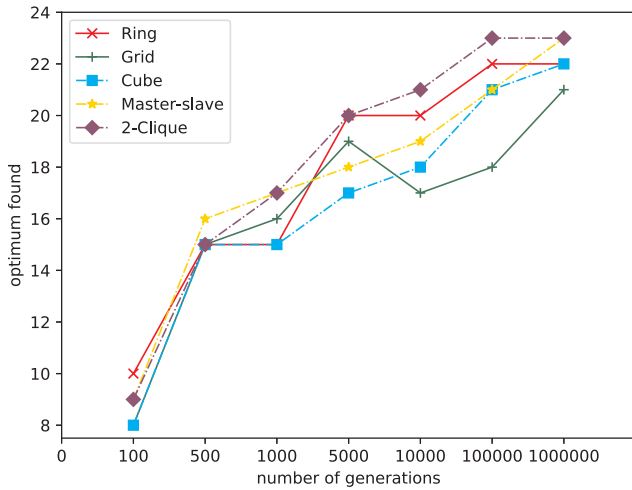
The effects of maximum number of generations on result quality.

# of Generations	u_120	u_250	u_500	u_1000	Data Set 1	Data Set 2	Data Set 3	Hard28
100	20	20	20	20	720	480	10	9
500	20	20	20	20	720	480	10	15
1000	20	20	20	20	720	480	10	14
5000	20	20	20	20	720	480	10	15
10,000	20	20	20	20	720	480	10	16
100,000	20	20	20	20	720	480	10	20
1,000,000	20	20	20	20	720	480	10	22

Table 6

Effects of topology with varying population sizes and number of generations on result quality. Pop. refers to population size and Gen. refers to number of generations.

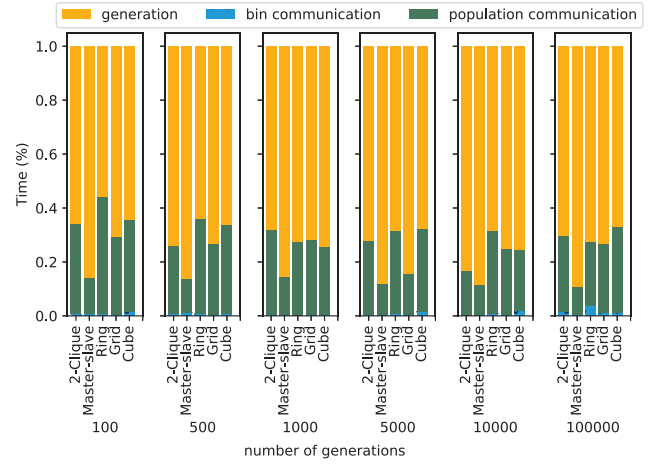
Topology	Pop = 500 Gen = 10,000		Pop = 1000 Gen = 10,000		Pop = 500 Gen = 100,000	
	Data Set 1	Hard28	Data Set 1	Hard28	Data Set 1	Hard28
Master-slave	720	19	720	18	720	21
OOPC	720	16	720	19	720	20
Ring	720	20	720	19	720	22
Grid	720	17	720	18	720	18
Cube	720	18	720	19	720	21
2-Clique	720	21	720	19	720	23
4-Clique	720	18	720	19	720	19

**Fig. 3.** The result performance of static communication topologies with varying number of generations.

do not require any communication. Consequently, finer grained analysis such as execution time breakdown on these datasets did not result in fruitful discussion. For this reason, findings on more broad and numeric discussions are presented for all of the aforementioned datasets and more detailed evaluations such as execution time breakdown analysis are only evaluated with the relatively challenging Hard28 dataset.

The baseline parameters for IPGGA that are adopted during this study is given in Table 2. On all experiments, each processor is given a fixed random seed so that the initial population remains the same through all experiments on each processor (However each processor is given a separate seed so that the initial populations on the processors may vary).

As an initial baseline, we conduct a series of preliminary experiments on sequential GGA (single island case). We investigate the effect of changing population size over result quality where maximum

**Fig. 4.** Execution time breakdown for static communication topologies with varying number of generations.

number of generations is set to 10,000 and report our findings in Table 3. The results show that, changing population size has no effect on the Falkenauer U and Scholl datasets, since all problem instances could be solved successfully. For Hard28 instances, for a single island, increasing population size has little effect on result quality given that a sufficient population size is selected. We also want to point out that the memory requirements and sequential run time for the GGA increases linearly with respect to the population size. Hence, we conclude that sequential GGA has a practical upper limit for the result performance depending on the population size.

5.1. Population size of islands

In sequential GGA, the population size is closely related with the search space exploration and exploitation. In order to measure the effects of island population on IPGGA result quality, we have set up an experiment. We have used the OOPC topology with 64 islands, where maximum number of generations set to 10,000, migration frequency to 0.1%, and assimilated the worst individuals during emigration. Table 4 presents our findings. Note that, we have not selected a fixed migration rate in our experiments; rather, we set it dynamically so that a set of best individuals proportional to the population size would be selected and used during the migration operation. The experiment validates the viability of the island parallel model, producing large improvements on result quality against its sequential counterpart.

It should also be noted here that, we are not attempting to compare either the result quality or the execution time of the parallel model with its sequential counterpart directly. Note that the population size numbers reported in Tables 3 and 4 are not compatible with each other and should not be used for a direct comparison. In fact, the total population size realized by the parallel model is much larger than the ones reported in the sequential experiments. This is due to the fact that both the total population size and number of generations used in most parallel evaluations would constitute to impractical sequential run parameters, which would render a comparison nonsensical. Consider the fact that using merely 500 as the population size in a parallel run with 64 processors would constitute to a total population size of 32,000 on a sequential run, which would have taxing memory and computational resource consumption for a sequential run. We also refrain ourselves from comparing the computation time of a parallel run with that of a sequential run, since any parallel model would be slower (in total computation time) than its sequential counterpart due to added communication and/or computation. Instead, we attempt to show the breakdown of computation time and communication time later in this section, providing insight on the scalability of a parallel model.

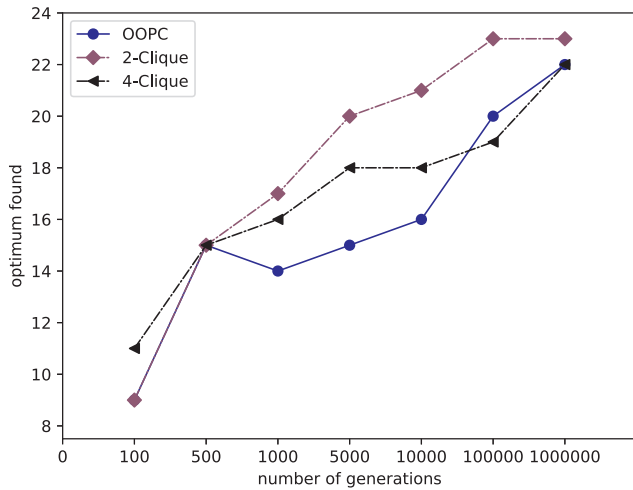


Fig. 5. The result performance of dynamic communication topologies with varying number of generations.

5.2. Number of generations

As a second baseline, we investigate the effects of different number of generations values over performance and present our results in Table 5. To this end, the population size of each island is set to 500, migration frequency to 10 generations per migration and OOPC communication topology is adopted during migration. Similar to the previous experiments, migration rate is set to 0.2% of local population size.

Even though number of generations to solve simpler problem instances (Falkenauer U and Scholl) is too small, the results indicate that allowing the algorithm to run longer achieves better results for the Hard28 instances. Only 9 of the 28 optimum instances could be solved within the first 100 generations, while up to 22 instances can be solved when the maximum number of generations is increased to 1,000,000.

For a better cross-comparison, we provide the result performances of all topologies with varying population sizes and maximum number of generations in Table 6. Results indicate that increasing population size adversely affects the result performance, whereas increasing the maximum number of generations has a consistent positive effect.

5.3. Static communication topologies

In Fig. 3, we provide the comparison of static communication topologies with a fixed population size of 500 and varying number of generations in terms of result quality over Hard28 instances. In our experiments, we used a fixed migration frequency of 10 generations per migration, and always assimilated the worst individuals. The migration rate is set to 0.2% of the local population size.

The comparison of Master-Slave and Ring topologies reveals the fact that Master-Slave topology achieves better result quality in fewer generations. Considering that each computational node only has a small pool of candidates in this experimental setting, this is a predictable outcome: maintaining population diversity in a computational node would be in a conflict with exploitation of best candidates since there is not enough space to accommodate both best solutions and a diverse population. Since Master-Slave topology has a much smaller diameter, propagation of candidate populations is much faster, i.e. exploitation of the search space is utilized. However, we should again point out the fact that Master-Slave topology is not ideal for large scale parallelism, due to the master node being a communication bottleneck.

Our empirical results also show that Hypercube topology also converges to similar results requiring more generations than the Ring topology. We observe that the result performance of the 2D Toroidal Grid topology achieves acceptable results faster, within a small number of generations, but failing to show the same improvement on the later

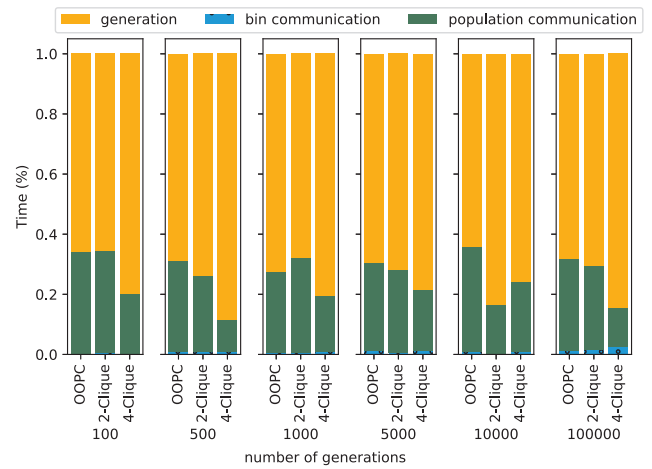


Fig. 6. Execution time breakdown of dynamic communication topologies with varying number of generations.

generations. We conclude that for static communication topologies there is no direct correlation between the result quality and the degree of connectivity of the nodes in the communication topology. Also note that Fig. 3 also includes the 2-Clique dynamic communication topology for two reasons. First, it provides a reference point in comparing static and dynamic communication topologies and second, due to this topology being the most restricted dynamic case, does not effect connectivity pattern during communication. The experimental results lead to the conclusion that for smaller number of generations Ring topology, and for larger number of generations Ring and 2-Clique topologies perform superior to other topologies in terms of result quality.

Fig. 4 provides a breakdown of the execution time of IPGGA with varying static communication topologies and number of generations averaged over execution times of all islands. In the figure, the execution time is divided into 3 phases: generation, bin communication, and population communication, and their contributions over the total execution time is reported. In the figure, the generation phase constitutes to all computational tasks such as fitness calculation and Darwinian selection operators, in addition to selecting best candidates among incoming populations and selecting which local population instances would be removed in order to make up space for the assimilated population; whereas bin and population communication phases constitute to additional costs due to parallelization. Here, the bin communication constitutes to selecting the immigrant population and their transmission times, and population communication constitutes to acquiring the immigrating population and assimilating them.

Conforming to the previous results in the literature, bin communication phase has an almost-negligible execution time. However, population communication may take from 10% to 40% of the total execution time which shows that, contrary to the reports in the literature when using large-scale parallelism, the assimilation operation in fact affects execution times considerably. Among all communication topologies Master-Slave topology has spent largest portion of the execution time on computational tasks validating that this topology would be ill-suited for high level of parallelism. Since in 2-Clique topology the

Table 7

The effect of migration rate on result quality.

Migration rate (% Population)	Data Set 1	Hard28
0.1	720	19
1	720	18
2	720	16
5	720	17
8	720	7
10	720	9

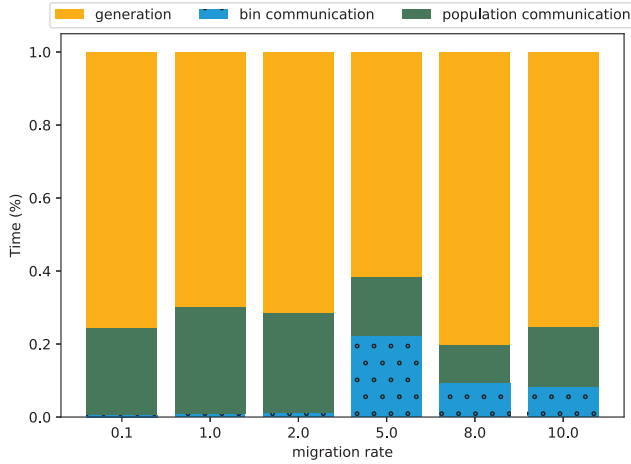


Fig. 7. Execution time breakdown of IPGGA with varying migration rates.

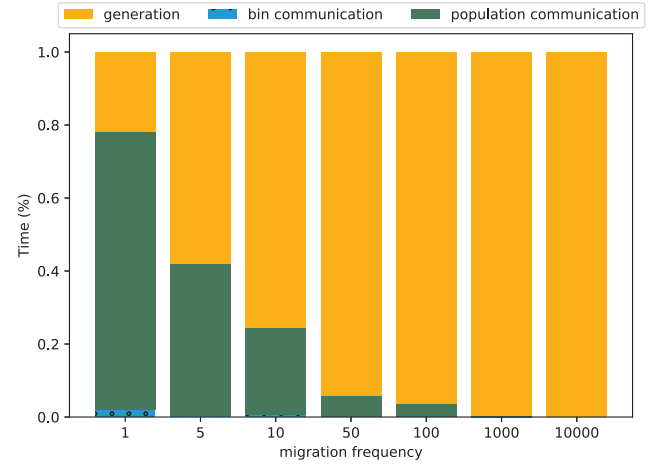


Fig. 8. Execution time breakdown for varying migration frequencies.

islands only communicate within isolated pairs, its computation-to-communication ratio is higher than that of a Master-Slave communication topology. In both Grid and Cube topologies, the communication volume is distributed roughly uniformly between islands and thus these topologies should incur similar communication costs as the 2-Clique topology. The results also indicate that these topologies in fact perform similar to the 2-Clique topology validating that these communication topologies have acceptable scaling.

5.4. Dynamic communication topologies

In Fig. 5, we provide the performance comparison of the dynamic communication topologies with a fixed population size of 500 on varying number of generations in terms of result quality on the Hard28 dataset. Similar to the experimental results for static communication topologies, we set the migration rate to 0.2% of the island population size, and always assimilated the worst individuals.

Experiments on dynamic communication topologies show that, the degree of communication per node has a direct impact on result quality. During the migration operation the results indicate that, when an island interacts with more neighbors, the poorer the result quality of the topology becomes. 2-Clique topology achieves the best overall result, 23 out of 28 optimal solutions for the Hard28 dataset. We would like to point out that, the 2-Clique topology is in fact the topology that promotes island isolation most among all topologies, while producing the best-known GGA results in literature. Based on these findings, we conclude that while having positive effects, migration should be performed with moderation and in isolation to preserve localized population diversity in order to achieve the best results. We also would like to point out the fact that none of the computational nodes in the dynamic communication topologies is a computational or communication bottleneck, and all nodes perform roughly the same amount of work and all topologies roughly have the same total communication volume at every generation. The performance results of dynamic communication topologies are also always either better or on-par with their static

Table 8

The effect of migration frequency on result quality.

Migration frequency (# Generations)	Data Set 1	Hard28
1	720	18
5	720	18
10	720	19
50	720	18
100	720	18
1000	720	19
10,000	720	19

Table 9

The effects of assimilation strategy on result quality.

Population	# of Generations	Assimilation strategy	Migration rate	Data Set 1	Hard28
500	500	Worst	1	720	15
500	500	Random	1	720	15
500	500	Best	1	720	14
500	500	Worst	10	720	15
500	500	Random	10	720	15
500	500	Best	10	720	14
500	10,000	Worst	1	720	16
500	10,000	Random	1	720	16
500	10,000	Best	1	720	17
500	10,000	Worst	10	720	16
500	10,000	Random	10	720	16
500	10,000	Best	10	720	15
1000	500	Worst	1	720	15
1000	500	Random	1	720	15
1000	500	Best	1	720	15
1000	500	Worst	10	720	16
1000	500	Random	10	720	15
1000	500	Best	10	720	15
1000	10,000	Worst	1	720	19
1000	10,000	Random	1	720	17
1000	10,000	Best	1	720	19
1000	10,000	Worst	10	720	18
1000	10,000	Random	10	720	17
1000	10,000	Best	10	720	18

counterparts, making them a more natural choice for a large-scale parallelism.

Fig. 6 reports the breakdown of the execution time of IPGGA with varying dynamic communication topologies and number of generations averaged over execution times of all islands. The results show that both OOPC and 2-Clique has very similar execution time breakdowns. Comparing the results in Figs. 4 and 6 show that OOPC communication topology also has very similar execution time breakdown to that of a ring communication topology. This is a foreseeable result due to similar characteristics of both topologies: OOPC actually runs a ring communication with varying island orderings at each communication step. Compared to other dynamic topologies, a larger proportion of the total execution time is spent on computational tasks using 4-Clique topology. We suspect that this phenomenon is due to a decreased cost of

assimilation operation on the 4-Clique topology, which is also indicated by the poor result quality for the aforementioned topology. (Note that the assimilation operation has the inherent cost of sorting the population with respect to fitness values to facilitate computations in the later generations, which may show variety depending on immigrant population characteristics.) As a summary, our results indicate that although having relatively worse result quality, 4-Clique communication topology has the highest potential for scaling in terms of parallel processing, while 2-Clique communication topology consistently outperforms other topologies.

5.5. Migration rate

Island parallel GGAs, provide a facility for manipulating the search space diversity via migration operators. Islands, by selectively sharing/exchanging a set of prospective candidate solutions among each other, could improve their chances for survival. The effects of migration heavily depend on some factors such as migration rate and frequency. As explained previously in Section 2, it is known that the migration can improve both solution time and quality considerably. However, there is no consensus in the literature on the ideal migration rate. Thus, we have also evaluated the effects of migration rate and frequency, providing our results.

Table 7 presents the observed effects of migration rate on result quality. We used OOPC topology with a population size of 1000 and maximum number of generations set to 10,000. The migration frequency was selected as one migration every 10 generations and always replaced the worst individuals as the assimilation strategy. The experimental results indicate that selection of migration rate has a large impact on the result quality. Smaller migration rates consistently achieve better results than higher migration rates. Conversely, when migration rates are higher, the result quality drops significantly.

Fig. 7 provides an execution time breakdown with varying migration rates. For all test cases, the computation-to-population communication phase ratio remain roughly constant, while bin communication phase shows variability. For smaller migration rates, the bin communication remains almost negligible. However, when migration rate is raised to 5% a sharp increase is observed, followed by a steady decline. This is due to a specific implementation detail regarding the messaging library (MPI) used in this framework: In the case that an inter-processor communication volume is smaller than a specified size, a single message is sent with MPI between the communicating peers. However, if communication volume between peers exceeds that specific size, the message need to be divided into multiple packages incurring an extra cost. Disregarding this overhead, the increase in migration rate linearly affects both computational and communication costs.

5.6. Migration frequency

Table 8 presents the effect of migration frequency. We used OOPC topology with a population size of 1000 and set the maximum number of generations to 10,000. The migration rate was selected as 0.1% of the island population size and worst individuals are replaced as the assimilation strategy. The experiment demonstrates that the selection of migration frequency has little impact on the number of optimum solutions found. Our results provide no conclusive insight on the ideal migration frequency, at least not a consistent one. Fig. 8 summarizes the execution time breakdown with varying migration frequencies. When migration frequency is low, i.e. communication is performed more frequently, communication time has a considerable effect on total execution time. As expected, with more practical migration frequency values, computation time dominates the total execution times and communication time become almost negligible.

5.7. Assimilation strategies

While examining the effects of migration, it is also important to analyze assimilation, i.e. which items to evict from the already existing solution set to make up room for the incoming population. In Table 9, we investigate the effects of assimilation strategies with varying population sizes, number of generations and migration rates for two relatively more challenging datasets: Data Set 1 and Hard28. The experimental results under different parameter settings indicate that, assimilating the worst individual in the population achieves more robust solutions than assimilating the best or a random individual. Among all eight settings, assimilating the worst individuals achieves the highest number of best solutions seven times, whereas both random and best assimilation strategies can only achieve four best solutions. These results show that an *elitist* approach for assimilation achieves best results, which is in concordance with other studies in the literature.

An interesting result of the evaluation of the assimilation strategies is that, although the elitist approach performs best, selecting the *best* individuals for assimilation outperform the random strategy consistently and even elitist approach in one of the cases. The two non-random assimilation strategies have only one common property: they affect the population diversity on a regular basis. While selecting the worst individuals for assimilation promotes best-performing chromosomes by regularly replicating them among islands, thus creating a higher selective pressure on these genes, this approach also regularly eliminates *not-so-well* performing genes, thus reducing the population diversity. On the other hand, selecting the best individuals for assimilation clearly just swaps best individuals on different islands, creating no additional selective pressure on well-performing genes. However,

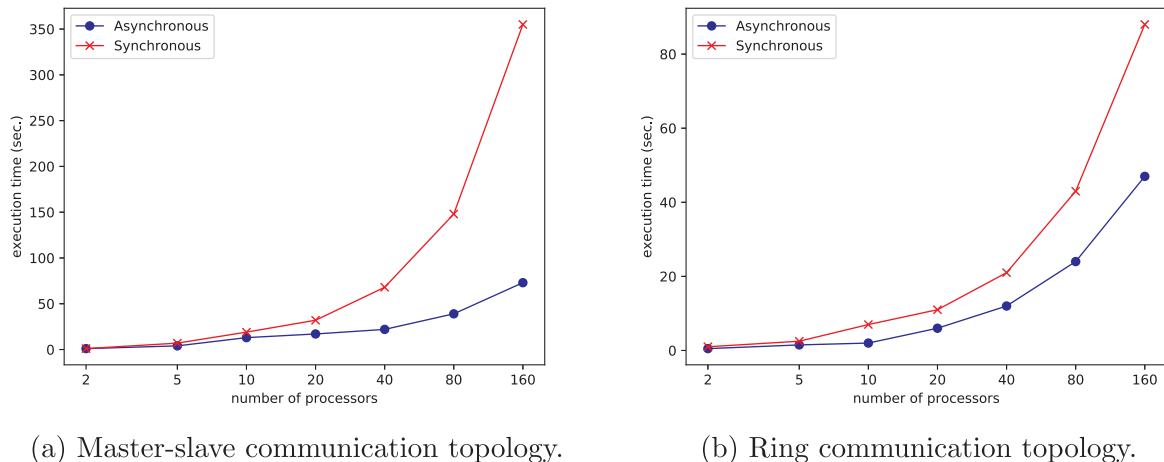


Fig. 9. Performance of two communication topologies with synchronous and asynchronous migration operators.

Table 10
Comparison of IPGGA result quality with other studies in the literature.

Study	u_120	u_250	u_500	u_1000	Data Set 1	Data Set 2	Data Set 3	Hard28
HGGA	18/20	18/20	20/20	20/20	–	–	–	0/28
BISON	–	–	–	–	290/323	234/244	10/10	0/28
BELOV	–	–	–	–	323/323	244/244	10/10	28/28
ARCFLOW	–	–	–	–	323/323	231/244	0/10	26/28
VPSOLVER	–	–	–	–	323/323	242/244	10/10	26/28
GGA-CGT	20/20	20/20	20/20	20/20	720/720	480/480	10/10	16/28
HI-BP	20/20	20/20	20/20	20/20	720/720	480/480	10/10	5/28
PERT-SAWMBS	20/20	20/20	20/20	20/20	720/720	480/480	10/10	5/28
EXON-MBS-BFD	20/20	20/20	17/20	18/20	667/720	412/480	8/10	5/28
IPGGA	20/20	20/20	20/20	20/20	720/720	480/480	10/10	23/28

since no population instance is actually eliminated, this approach can be viewed as the strategy that preserves the population diversity best. It is very clear from this set of experiments that, preserving diversity has a large impact for a genetic algorithm, comparable to that of promoting best chromosomes.

5.8. Synchronous vs. asynchronous communication

The final issue on an island parallel GGA is synchronization. After each migration operation, there can be two possible execution strategies: forcing synchronization after each generation or allowing an asynchronous communication. A synchronous generation operation constitutes to having the computational nodes wait until all processors finish the migration operations. Conversely, an asynchronous generation operation constitutes to allowing computational nodes to start their next generation with incomplete migration information, i.e., disregarding any late coming immigrants. The literature indicates that, synchronous case is more preferable to the latter in terms of result quality, though an asynchronous communication would be faster.

In this section, we present the overhead of synchronous and asynchronous communication on different communication topologies. To this end, we select and evaluate two opposite static communication topologies with varying degree of connectivity and graph diameter: Master-Slave and Ring communication topologies. In our experiments, we select migration rate as 500 population instances, migration frequency as 1 migration per generation, and the maximum number of the generations as 100,000. To provide clearer results, we have also evaluated our experiments with a larger number of computational nodes: 160 nodes. When the number of processors is 160, the communication cost of the synchronous Master-Slave topology has increased 364.6% with respect to its asynchronous counterpart. For the Ring topology, the overhead of synchronous communication is observed to be 80.4% more than its asynchronous version (see Fig. 9a and b).

Master-Slave and Ring topologies both potentially send/receive a total of $2 \times \text{number of processors} \times \text{generations}$ population instances among islands during these experiments regardless of communication being synchronous or asynchronous. Comparison of both topologies for both synchronous and asynchronous communication reveal similar results. If allowed similar communication times, the Ring topology can send/receive 59.2% more instances than the Master-Slave topology during a GA run. Thus, the performance of Ring topology is observed to be better than the Master-Slave topology, where synchronous communication always performs better. With asynchronous messaging, the Ring topology can finish the migration operation 94.3% earlier than Master-Slave topology.

The results indicate that, using a synchronous communication create larger overhead than its asynchronous communication. Second, synchronous communication is more effected from a bottleneck on one or more computational nodes. Finally, we conclude that overhead of communication diameter has high impact on execution time. Even though number of sent/received messages does not differ for both

topologies, the Ring topology completes its migration operation faster than the Master-Slave communication topology due to its smaller diameter. As a result, it is possible to conclude that overhead produced by asynchronous messaging on a smaller communication diameter has lower impact than synchronous messaging on a larger communication diameter.

5.9. Comparison with the literature

In order to evaluate the result performance of the proposed IPGGA, we select the best settings for the proposed algorithm and compare our results with several studies in the literature. Table 10 presents a detailed view on the result performance of these algorithms over the datasets that have been evaluated in this work. Please note that, the dashed lines in the table cells indicate that the particular algorithm has not been tested with the particular dataset in the original work, or in any other study that replicate the aforementioned work. We have also selected a wide range of studies for this comparison: HGGA (Falkenauer, 1996a), GGA-CGT (Quiroz-Castellanos et al., 2015), HI-BP (Alvim et al., 2004), and PERT-SAWMBS (Fleszar & Charalambous, 2011) algorithms are selected as well-known sequential methods in the literature for solving the BPP, whereas BISON (Scholl et al., 1997), BELOV (Belov & Scheithauer, 2002), ARCFLOW (Valério De Carvalho, 1999), and VPSOLVER (Brandão & Pedroso, 2016) are based on exact methods for the BPP. For the purpose of comparing our results to that of an exact method, we restrict the execution time of the exact algorithm with a reasonable time limit (up to 10 min).² We have also compared IPGGA with other GGA-based methods such as GGA-CGT (Quiroz-Castellanos et al., 2015) which is a sequential GGA-based method and EXON-MBS-BFD (Dokeroglu & Cosar, 2014) which is a parallel implementation of GGA.

The comparison of the result quality of IPGGA and other studies show that the proposed IPGGA perform superior to any other heuristic in the literature. Moreover, compared to exact algorithms, IPGGA consistently provide results that are on the better spectrum of the comparison. Here, we would also like to note that, in addition to result quality, the proposed IPGGA provide a scalable framework for the BPP and have additional merits for larger and challenging datasets.

6. Conclusion

In this study, we introduce a scalable parallel implementation of a grouping genetic algorithm using an island parallel model for the solution of the 1DBPP. We evaluate our model on three data sets of problem instances. While perfectly solving the simpler dataset instances (Falkenauer U and Scholl), our model could manage to solve 23 out of 28 problem instances of Hard28 dataset optimally while producing only 5 extra bins in the remaining 5 instances, which is the best performance

² A more thorough analysis of exact algorithms can be found in Delorme, Iori, and Martello (2016).

of a GA model to date in the literature. We also provide an in-depth analysis of the parametrization of the island parallel model.

By evaluating various GGA parameters in this work, it is possible to derive several conclusions for a successful parallel GGA framework. As for the sequential GA parameters, our results conform with the previous literature; increasing the maximum number of generations always lead to better results, while increasing population size does not necessarily guarantee a better quality. Since population size is directly proportional to the computation time, it would be wiser to select a modest population size in a successful GGA. As for the parallel GGA parameters, our study led to more interesting results. Selecting migration rate as small as possible seem to be a viable decision in terms of total execution time, and the experiments on the result quality also show that small migration rates achieve better results. For migration frequency, although slight improvements can be observed at higher frequencies, the communication cost of the parallel framework also decreases proportionally. Thus, lower migration frequencies should be avoided due to increased communication costs for little-to-no benefit. Lastly, as assimilation strategies, our experiments showed that although best results are achieved by selecting the worst population instances for assimilation, selecting the best population instances for assimilation still achieves better results than a random selection. Our conclusion is that, assimilation strategy is an integral decider on population diversity and more evaluation is required in order to infer the underlying mechanics of assimilation.

A noticeable contribution of this work is the introduction and evaluation of dynamic communication topologies which are more suitable parallelization alternatives for highly distributed computing. We feel that our results on dynamic topologies also provide new insight on hierarchical island communication patterns. Our results show that among all communication topologies, a 2-Clique communication topology performs best in terms of result performance, where islands are separated into completely isolated pairs, namely archipelagos. The above analysis provides clues that there is still room for understanding the mechanics of parallel genetic algorithms, and via careful engineering of problem- and architecture-specific communication topologies, it is possible that the exploitation and exploration trade-off of GGA can be further manipulated in order to achieve better and faster results.

The work presented here can be extended in several ways. First, we believe that for large scale distributed genetic algorithm frameworks, dividing islands into hierarchically isolated archipelagos may have a large potential in terms of result quality. In this work, we have only examined 2-level communication hierarchies. Evaluating multi-level hierarchies can be an obvious direction for future research. Second, genetic algorithms enjoy very similar benefits with neural network algorithms: Both are iterative in nature and during each iteration computational bottleneck is mainly localized operations. Recent research on neural networks showed that SIMD multicomputer architectures such as FPGAs (Field Gate Programming Arrays) can achieve super-scalar speedup. Integrating GA and FPGAs may provide huge benefits for the community in terms of computational potential.

Acknowledgements

We would like to thank Marcela Quiroz-Castellanos, Laura Cruz-Reyes, Jose Torres-Jimenez, Claudia Gomez S., Hector J. Fraire Huacuja, Adriana C.F. Alvim for the support they have provided during our study.

Appendix A. Supplementary material

Supplementary data associated with this article can be found, in the online version, at <http://dx.doi.org/10.1016/j.cie.2018.08.021>.

References

- Alba, E., & Troya, J. M. (1999). A survey of parallel distributed genetic algorithms. *Complexity*, 4(4), 31–52.
- Alvim, A. C. F., Ribeiro, C. C., Glover, F., & Aloise, D. J. (2004). A hybrid improvement heuristic for the one-dimensional bin packing problem. *Journal of Heuristics*, 10(2), 205–229.
- Andre, D., & Koza, J. R. (1996). Parallel genetic programming: A scalable implementation using the transputer network architecture. *Advances in genetic programming 2* (pp. 317–337). MIT Press.
- Arbib, C., & Marinelli, F. (2017). Maximum lateness minimization in one-dimensional bin packing. *Omega*, 68, 76–84.
- Bein, W., Correa, J. R., & Han, X. (2008). A fast asymptotic approximation scheme for bin packing with rejection. *Theoretical Computer Science*, 393(1), 14–22.
- Belov, G., & Scheithauer, G. (2002). A cutting plane algorithm for the one-dimensional cutting stock problem with multiple stock lengths. *Journal of Operational Research*, 141(2), 274–294.
- Belov, G., Scheithauer, G., & Mukhacheva, E. A. (2008). One-dimensional heuristics adapted for two-dimensional rectangular strip packing. *Journal of the Operational Research Society*, 59(6), 823–832.
- Berghammer, R., & Reuter, F. (2003). A linear approximation algorithm for bin packing with absolute approximation factor 32. *Science of Computer Programming*, 48(1), 67–80.
- Beyaz, M., Dokeroglu, T., & Cosar, A. (2015). Robust hyper-heuristic algorithms for the offline oriented/non-oriented 2d bin packing problems. *Applied Soft Computing*, 36, 236–245.
- Błażewicz, J., & Ecker, K. H. (1983). A linear time algorithm for restricted bin packing and scheduling problems. *Operations Research Letters*, 2(2), 80–83.
- Boyar, J., Kamali, S., Larsen, K. S., & López-Ortiz, A. (2016). Online bin packing with advice. *Algorithmica*, 74(1), 507–527.
- Bożejko, W., Uchroński, M., & Wodecki, M. (2010). Parallel hybrid metaheuristics for the flexible job shop problem. *Computers & Industrial Engineering*, 59(2), 323–333.
- Brandão, F., & Pedro Pedrosa, J. (2016). Bin packing and related problems. *Computers and Operations Research*, 69(C), 56–67.
- Buljuba, M., & Vasquez, M. (2016). Consistent neighborhood search for one-dimensional bin packing and two-dimensional vector packing. *Computers and Operations Research*, 76, 12–21.
- Cantu-Paz, E. (2000). *Efficient and accurate parallel genetic algorithms*. Springer Science & Business Media.
- Christensen, H. I., Khan, A., Pokutta, S., & Tetali, P. (2017). Approximation and online algorithms for multidimensional bin packing: A survey. *Computer Science Review*, 24, 63–79.
- Coffman, E. G., Jr., Csirik, J., Galambos, G., Martello, S., & Vigo, D. (2013). *Bin packing approximation algorithms: Survey and classification*. New York, New York, NY: Springer455–531.
- Coffman, E. G., Galambos, G., Martello, S., & Vigo, D. (1999). Bin packing approximation algorithms: Combinatorial analysis. *Handbook of combinatorial optimization* (pp. 151–207). Springer.
- Coffman, E. G., Jr., Garey, M. R., & Johnson, D. S. (1996). Approximation algorithms for bin packing: A survey. *Approximation algorithms for NP-hard problems* (pp. 46–93). .
- Correa, J. R., & Epstein, L. (2008). Bin packing with controllable item sizes. *Information and Computation*, 206(8), 1003–1016.
- de Almeida, R., & Steiner, M. T. A. (2015). Resolution of 1-d bin packing problem using augmented neural networks and minimum bin slack. *2015 Latin America congress on computational intelligence (LA-CCI)* (pp. 1–6). .
- Delorme, M., Iori, M., & Martello, S. (2016). Bin packing and cutting stock problems: Mathematical models and exact algorithms. *European Journal of Operational Research*, 255(1), 1–20.
- Dokeroglu, T., & Cosar, A. (2014). Optimization of one-dimensional bin packing problem with island parallel grouping genetic algorithms. *Computers & Industrial Engineering*, 75, 176–186.
- Dokeroglu, T., & Cosar, A. (2016). A novel multistart hyper-heuristic algorithm on the grid for the quadratic assignment problem. *Engineering Applications of Artificial Intelligence*, 52, 10–25.
- Epstein, L. (2010). Bin packing with rejection revisited. *Algorithmica*, 56(4), 505–528.
- Epstein, L., & Levin, A. (2012). Bin packing with general cost structures. *Mathematical Programming*, 132(1), 355–391.
- Falkenauer, E. (1994). A new representation and operators for gas applied to grouping problems. *Evolutionary Computation*, 2(2), 123–144.
- Falkenauer, E. (1996a). *The grouping genetic algorithm*. Boston, MA: Springer US249–265.
- Falkenauer, E. (1996b). A hybrid grouping genetic algorithm for bin packing. *Journal of heuristics*, 2(1), 5–30.
- Falkenauer, E., & Delchambre, A. (1992). A genetic algorithm for bin packing and line balancing. *Proceedings of the 1992 IEEE international conference on robotics and automation* (pp. 1186–1192). .
- Fernández, F., Galeano, G., & Gómez, J. A. (2002). *Comparing synchronous and asynchronous parallel and distributed genetic programming models*. Berlin, Heidelberg: Springer Berlin Heidelberg326–335.
- Fernández, A., Gil, C., Baños, R., & Montoya, M. G. (2013). A parallel multi-objective algorithm for two-dimensional bin packing with rotations and load balancing. *Expert Systems with Applications*, 40(13), 5169–5180.
- Fernández, F., Tomassini, M., Punch, W. F., & Sánchez, J. M. (2000). *Experimental study of multipopulation parallel genetic programming*. Berlin, Heidelberg: Springer Berlin Heidelberg283–293.
- Fleszar, K., & Charalambous, C. (2011). Average-weight-controlled bin-oriented heuristics

- for the one-dimensional bin-packing problem. *European Journal of Operational Research*, 210(2), 176–184.
- Fleszar, K., & Hindi, K. S. (2002). New heuristics for one-dimensional bin-packing. *Computers & Operations Research*, 29(7), 821–839.
- Gary, M. R., & Johnson, D. S. (1979). *Computers and intractability: A guide to the theory of NP-completeness*. New York: WH Freeman and Company.
- Grefenstette, J. J., Gopal, R., Rosmaita, B. J., & Gucht, D. V. (1985). Genetic algorithms for the traveling salesman problem. *Proceedings of the 1st international conference on genetic algorithms* (pp. 160–168).
- Holland, J. H. (1992). *Adaptation in natural and artificial systems: An introductory analysis with applications to biology, control, and artificial intelligence*. MIT Press.
- Johnson, D. S., Demers, A., Ullman, J. D., Garey, M. R., & Graham, R. L. (1974). Worst-case performance bounds for simple one-dimensional packing algorithms. *SIAM Journal on Computing*, 3(4), 299–325.
- Juille, H., & Pollack, J. B. (1995). Parallel genetic programming and fine-grained simd architecture. *Working notes for the AAAI symposium on genetic programming* (pp. 31–37).
- Konfrst, Z. (2004). Advances, computing trends, applications and perspectives. *18th International parallel and distributed processing symposium, 2004. Proceedings* (pp. 162).
- Kröger, B., Schwenderling, P., & Vornberger, O. (1990). Parallel genetic packing of rectangles. *International conference on parallel problem solving from nature* (pp. 160–164).
- Leinberger, W., Karypis, G., & Kumar, V. (1999). Multi-capacity bin packing algorithms with applications to job scheduling under multiple constraints. *Proceedings of the 1999 international conference on parallel processing* (pp. 404).
- Liu, Y. Y., & Wang, S. (2015). A scalable parallel genetic algorithm for the generalized assignment problem. *Parallel computing*, 46(C), 98–119.
- López-Camacho, E., Terashima-Marín, H., Ochoa, G., & Conant-Pablos, S. E. (2013). Understanding the structure of bin packing problems through principal component analysis. *International Journal of Production Economics*, 145(2), 488–499.
- Luque, G., & Alba, E. (2011). *Parallel genetic algorithms: Theory and real world applications*. Springer.
- Maiza, M., Radjef, M. S., & Sais, L. (2016). Efficient lower bounds for packing problems in heterogeneous bins with conflicts constraint. *Intelligent mathematics II: Applied mathematics and approximation theory* (pp. 263–270). Springer.
- Martello, S., & Toth, P. (1990). *Knapsack problems: Algorithms and computer implementations*. John Wiley Sons, Inc.
- Mitchell, M. (1998). *An introduction to genetic algorithms*. MIT Press.
- Nalepa, J., & Blocho, M. (2015). Co-operation in the parallel memetic algorithm. *International Journal of Parallel Programming*, 43(5), 812–839.
- Ozcan, S. O., Dokeroglu, T., Cosar, A., & Yazici, A. (2016). A novel grouping genetic algorithm for the one-dimensional bin packing problem on gpu. *International symposium on computer and information sciences* (pp. 52–60).
- Pargas, R. P., & Jain, R. (1993). A parallel stochastic optimization algorithm for solving 2d bin packing problems. *Artificial intelligence for applications, 1993. Proceedings, Ninth conference on* (pp. 18–25).
- Quiroz-Castellanos, M., Cruz-Reyes, L., Torres-Jimenez, J., Gómez, C., Huacuja, H. J. F., & Alvim, A. C. F. (2015). A grouping genetic algorithm with controlled gene transmission for the bin packing problem. *Computers & Operations Research*, 55, 52–64.
- Radcliffe, N. J. (1991). Equivalence class analysis of genetic algorithms. *Complex Systems*, 5(2), 183–205.
- Ribeiro, C. C., & Hansen, P. (2002). *Essays and surveys in metaheuristics*. Norwell, MA, USA: Kluwer Academic Publishers.
- Ruciński, M., Izzo, D., & Biscani, F. (2010). On the impact of the migration topology on the island model. *Parallel Computing*, 36(10), 555–571.
- Scholl, A., Klein, R., & Jürgens, C. (1997). Bison: A fast hybrid procedure for exactly solving the one-dimensional bin packing problem. *Computers & Operations Research*, 24(7), 627–645.
- Schwehm, M. (1996). *Parallel population models for genetic algorithms*.
- Segura, C., Segredo, E., & León, C. (2011). Parallel island-based multiobjective memetic algorithms for a 2d packing problem. *Proceedings of the 13th annual conference on genetic and evolutionary computation* (pp. 1611–1618).
- Stawowy, A. (2008). Evolutionary based heuristic for bin packing problem. *Computers & Industrial Engineering*, 55(2), 465–474.
- Stolyar, A. L., & Zhong, Y. (2016). A greedy randomized algorithm achieving sublinear optimality gap in a dynamic packing model. *2016 54th annual Allerton conference on communication, control, and computing (Allerton)* (pp. 319–326).
- Thomas, J., & Chaudhari, N. S. (2014). Design of efficient packing system using genetic algorithm based on hyper heuristic approach. *Advances in Engineering Software*, 73, 45–52.
- Tomassini, M. (1999). *Parallel and distributed evolutionary algorithms: A review*.
- Tosun, U., Dokeroglu, T., & Cosar, A. (2013). A robust island parallel genetic algorithm for the quadratic assignment problem. *International Journal of Production Research*, 51(14), 4117–4133.
- Tufts, P. (1995). Parallel case evaluation for genetic programming. *1993 Lectures in complex systems* (pp. 591–596). Addison-Wesley.
- Ullman, J. D. (1971). The performance of a memory allocation algorithm, Technical report (Volume 100), Princeton University, Department of Electrical Engineering. Computer Science Laboratory.
- Valério De Carvalho, J. M. (1999). Exact solution of bin-packing problems using column generation and branch-and-bound. *Annals of Operations Research*, 86, 629–659.
- Zendaoui, Z., & Layeb, A. (2016). Adaptive cuckoo search algorithm for the bin packing problem. *Modelling and implementation of complex systems* (pp. 107–120). Springer.